

BLOCKCHAIN LABORATORY

Experiment No. 2

Create a Blockchain using Python

Name: Nikhil Sunchu

Roll No.: 31

Class/section D20A

Date _____

1. Aim

Create a Blockchain using Python.

2. Theory

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

3. Tools and Libraries Used

- Python
- Python libraries: `datetime`, `hashlib`, `json`
- Flask (web API framework)

4. Code Overview

4.1. 1. Importing Required Libraries

- `datetime` → to timestamp each block
- `hashlib` → to generate SHA-256 hashes
- `json` → to encode blocks as JSON for hashing

- Flask → to run the blockchain API server

4.2. 2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

4.3. 3. Flask Web API

Provides endpoints:

- /mine_block → mine a new block
- /get_chain → fetch full blockchain
- /is_valid → verify blockchain

5. Complete Code

```
1 # -----
2 # Module 1 - Create a Simple Blockchain (Fully Commented)
3 # -----
4 import datetime
5 import hashlib
6 import json
7 from flask import Flask, jsonify, request
8
9 # -----
10 # Part 1 - Building the Blockchain
11 # -----
12 class Blockchain:
13
14     def __init__(self):
15         # Store all mined blocks and pending transactions
16         self.chain = []
17         self.transactions = []
18
19         # Create the genesis block
20         self.create_block(
```

```
21         proof=1,
22         previous_hash="0",
23         transactions=[]
24     )
25
26     def create_block(self, proof, previous_hash, transactions):
27         # Build a new block with the current timestamp and transaction list
28         block = {
29             "index": len(self.chain) + 1,
30             "timestamp": str(datetime.datetime.now()),
31             "transactions": transactions,
32             "proof": proof,
33             "previous_hash": previous_hash
34         }
35         self.chain.append(block)
36         return block
37
38     def get_previous_block(self):
39         # Return the latest mined block
40         return self.chain[-1]
41
42     def hash(self, block):
43         # Convert block data to a hash for integrity checking
44         encoded_block = json.dumps(
45             block,
46             sort_keys=True
47         ).encode()
48         return hashlib.sha256(encoded_block).hexdigest()
49
50     def proof_of_work(self, previous_proof):
51         # Find a number whose hash starts with 000 (mining challenge)
52         new_proof = 1
53         while True:
54             hash_operation = hashlib.sha256(
55                 str(new_proof ** 2 - previous_proof ** 2).encode()
56             ).hexdigest()
57             if hash_operation[:3] == "000":
58                 return new_proof
59             new_proof += 1
60
61     def is_chain_valid(self, chain):
```

```
62     # Verify all hashes and proof-of-work values in the chain
63     previous_block = chain[0]
64     block_index = 1
65     while block_index < len(chain):
66         block = chain[block_index]
67         if block["previous_hash"] != self.hash(previous_block):
68             return False
69
70         previous_proof = previous_block["proof"]
71         proof = block["proof"]
72         hash_operation = hashlib.sha256(
73             str(proof ** 2 - previous_proof ** 2).encode()
74         ).hexdigest()
75         if not hash_operation.startswith("000"):
76             return False
77
78         previous_block = block
79         block_index += 1
80     return True
81
82 def create_transaction(self, sender, receiver, amount):
83     # Store a new pending transaction before the next block is mined
84     transaction = {
85         "sender": sender,
86         "receiver": receiver,
87         "amount": amount
88     }
89     self.transactions.append(transaction)
90     return len(self.chain) + 1
91
92 def mine_block(self):
93     # Mine a new block only when at least one transaction is pending
94     if not self.transactions:
95         return None
96
97     previous_block = self.get_previous_block()
98     previous_proof = previous_block["proof"]
99     transactions = self.transactions
100     self.transactions = []
101
102     proof = self.proof_of_work(previous_proof)
```

```
103     previous_hash = self.hash(previous_block)
104     block = self.create_block(
105         proof=proof,
106         previous_hash=previous_hash,
107         transactions=transactions
108     )
109     return block
110
111
112 # -----
113 # Part 2 - Mining our Blockchain (Flask API)
114 # -----
115 app = Flask(__name__)
116
117 # Create a blockchain instance for the API server
118 blockchain = Blockchain()
119
120 # -----
121 # API Endpoints
122 # -----
123 @app.route("/get_chain", methods=["GET"])
124 def get_chain():
125     # Return the complete blockchain and its current length
126     response = {
127         "chain": blockchain.chain,
128         "length": len(blockchain.chain)
129     }
130     return jsonify(response), 200
131
132
133 @app.route("/add_transaction", methods=["POST"])
134 def add_transaction():
135     # Accept a JSON payload and queue a new transaction for mining
136     data = request.get_json()
137     sender = data["sender"]
138     receiver = data["receiver"]
139     amount = data["amount"]
140     block_index = blockchain.create_transaction(sender, receiver, amount)
141     response = {
142         "message": "Transaction added successfully",
143         "block_index": block_index,
```

```
144         "transaction": {
145             "sender": sender,
146             "receiver": receiver,
147             "amount": amount
148         }
149     }
150     return jsonify(response), 201
151
152
153 @app.route("/mine_block", methods=["GET"])
154 def mine_block():
155     # Mine the pending transactions into a new block
156     block = blockchain.mine_block()
157     if block is None:
158         return jsonify({
159             "message": "No transactions available. Block not mined."
160         }), 400
161     response = {
162         "message": "Block mined successfully.",
163         "index": block["index"],
164         "timestamp": block["timestamp"],
165         "transactions": block["transactions"],
166         "proof": block["proof"],
167         "previous_hash": block["previous_hash"]
168     }
169     return jsonify(response), 200
170
171
172 @app.route("/is_valid", methods=["GET"])
173 def is_valid():
174     # Check whether the blockchain has been tampered with
175     valid = blockchain.is_chain_valid(blockchain.chain)
176     if valid:
177         response = {"message": "Blockchain is valid."}
178     else:
179         response = {"message": "Blockchain is not valid."}
180     return jsonify(response), 200
181
182
183 # -----
184 # Run the Flask app
```

```
185 # -----  
186 if __name__ == "__main__":  
187     app.run(host="127.0.0.1", port=5000, debug=True)
```

6. Output

6.1. Fig. 1 – Get the Initial Chain with Genesis Block

```
~ > curl http://127.0.0.1:5000/get_chain  
{  
  "chain": [  
    {  
      "index": 1,  
      "previous_hash": "0",  
      "proof": 1,  
      "timestamp": "2026-09-01 19:51:02.844076",  
      "transactions": []  
    }  
  ],  
  "length": 1  
}
```

6.2. Fig. 2 – Add a Transaction

```
~ > curl -X POST http://127.0.0.1:5000/add_transaction \\  
-H "Content-Type: application/json" \\  
-d '{"sender": "Alice", "receiver": "Bob", "amount": 100}'  
{  
  "block_index": 2,  
  "message": "Transaction added successfully",  
  "transaction": {  
    "amount": 100,  
    "receiver": "Bob",  
    "sender": "Alice"  
  }  
}
```

6.3. Fig. 3 – Mine the New Block with the Newly Added Transaction

```
~ > curl http://127.0.0.1:5000/mine_block
{
  "index": 2,
  "message": "Block mined successfully.",
  "previous_hash": "7c4f371a5efef076c8f34739c95482228b5c8ae66e5d63e87d80fd3b1ee58718",
  "proof": 533,
  "timestamp": "2026-09-01 19:52:19.368352",
  "transactions": [
    {
      "amount": 100,
      "receiver": "Bob",
      "sender": "Alice"
    }
  ]
}
```

6.4. Fig. 4 – Get the Updated Chain with Mined Block

```
~ > curl http://127.0.0.1:5000/get_chain
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-01 19:51:02.844076",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "7c4f371a5efef076c8f34739c95482228b5c8ae66e5d63e87d80fd3b1ee58718",
      "proof": 533,
      "timestamp": "2026-09-01 19:52:19.368352",
      "transactions": [
        {
          "amount": 100,
          "receiver": "Bob",
          "sender": "Alice"
        }
      ]
    }
  ],
  "length": 2
}
```

6.5. Fig. 5 – Add Two New Transactions

```
~ > curl -X POST http://127.0.0.1:5000/add_transaction \
-H "Content-Type: application/json" \
-d '{"sender": "Bob", "receiver": "Charlie", "amount": 50}'
{
  "block_index": 3,
  "message": "Transaction added successfully",
  "transaction": {
    "amount": 50,
    "receiver": "Charlie",
    "sender": "Bob"
  }
}
~ > curl -X POST http://127.0.0.1:5000/add_transaction \
-H "Content-Type: application/json" \
-d '{"sender": "Charlie", "receiver": "David", "amount": "25"}'
{
  "block_index": 3,
  "message": "Transaction added successfully",
  "transaction": {
    "amount": "25",
    "receiver": "David",
    "sender": "Charlie"
  }
}
```

6.6. Fig. 6 – Mine a New Block with the Two Newly Added Transactions

```
~ > curl http://127.0.0.1:5000/mine_block
{
  "index": 3,
  "message": "Block mined successfully.",
  "previous_hash": "def5c1af8571a69cef78a7e59ce7786b68d52448dff52bb884bfd8d5ad91cc0",
  "proof": 11028,
  "timestamp": "2026-09-01 20:02:43.116375",
  "transactions": [
    {
      "amount": 50,
      "receiver": "Charlie",
      "sender": "Bob"
    },
    {
      "amount": "25",
      "receiver": "David",
      "sender": "Charlie"
    }
  ]
}
```

6.7. Fig. 7 – Get the Updated Chain with Three Blocks

```
~ > curl http://127.0.0.1:5000/get_chain
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-01 20:00:04.173384",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "9fdf7a4ee2b2d686e4e433d3bf82391731992c6c18e778ed7a3d552a42389738",
      "proof": 533,
      "timestamp": "2026-09-01 20:00:26.861503",
      "transactions": [
        {
          "amount": 100,
          "receiver": "Bob",
          "sender": "Alice"
        }
      ]
    },
    {
      "index": 3,
      "previous_hash": "def5c1af8571a69cef78a7e59ce7786b68d52448dff52bb884bdfd8d5ad91cc0",
      "proof": 11028,
      "timestamp": "2026-09-01 20:02:43.116375",
      "transactions": [
        {
          "amount": 50,
          "receiver": "Charlie",
          "sender": "Bob"
        },
        {
          "amount": 25,
          "receiver": "David",
          "sender": "Charlie"
        }
      ]
    }
  ],
  "length": 3
}
```

6.8. Fig. 8 – Validate the Chain

```
~ > curl http://127.0.0.1:5000/is_valid
{
  "message": "Blockchain is valid."
}
```

7. Conclusion

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles – decentralization, immutability, and consensus – in a simplified environment, providing a strong foundation for building more advanced blockchain systems.